

Jenkins Basics

Automation Server Guide: CI/CD Principles & Core Installation

Objective & Scope:

- Learn the basics of Jenkins Automation Server, its standard architecture, and layout.
- Learn about the commonly used CI/CD deployment workflows and practical core pipeline methods with Jenkins.
- Install and set up a live Jenkins server instance sequentially on your machine environment.

1. Introduction to Jenkins

Jenkins is an open-source automation server used extensively to automate diverse software development tasks such as building, testing, and deploying foundational applications. It stands as one of the most widely implemented ecosystems globally for enforcing Continuous Integration (CI) and Continuous Delivery/Deployment (CD) operations.

Key Features & Capabilities

- **Open-source and Free:** Maintained by an active community with no licensing overheads.
- **Web-based Dashboard:** Offers an intuitive layout for managing tasks, monitoring builds, and tracking logs.
- **Extensive Plugin Ecosystem:** Supports hundreds of plugins to easily extend platform functionality.
- **Source Control Integration:** Seamlessly connects with popular modern VCS providers like GitHub, GitLab, and Bitbucket.
- **End-to-End Automation:** Handles continuous automated compilation, test execution, and release rollouts.
- **Distributed Build Engine:** Supports distributed processing configurations leveraging master-agent (node) allocations.
- **Pipeline-as-Code:** Allows institutionalization of workflows cleanly defined using custom standard Jenkinsfile scripts.

2. Jenkins Architecture

To reliably coordinate scaled build structures, Jenkins relies on a highly cohesive decentralized orchestration architecture composed of four core pillars:

Architectural Component	Core System Responsibility
Controller (Master)	Responsible for central orchestration. It manages jobs, coordinates schedules, acts as the primary configuration repository, and directly controls connected processing agents.
Agent (Node)	Dedicated executor resources optimized to pick up and process specific build, test, compilation, or deployment workloads dispatched by the controller.
Plugins	Modular extension packages that enrich Jenkins with custom tool integrations, authentication extensions, cloud provisioning interfaces, and modern utility hooks.
Pipeline	Constructs that cleanly define the overall multi-stage CI/CD workflow lifecycle via software development definitions stored directly alongside source code (Jenkinsfile).

3. CI/CD Methodology & Workflow

Continuous Integration (CI)

Continuous Integration is the structural DevOps practice of frequently merging active source code modifications into a shared repository. Each push automatically triggers a remote build and execution of a localized test suite to capture regression errors quickly.

Standard CI Visual Workflow Loop:

Developer Work → Code Push to GitHub → Jenkins Hook Trigger → Source Build → Automated Testing → Matrix Reporting

Continuous Delivery vs. Continuous Deployment (CD)

Continuous Delivery guarantees that validated code versions are structurally packaged and always maintained in a deployable status for quick manual releases. Continuous Deployment expands this paradigm further by automatically routing every successful master branch commit directly into active production environments without manual confirmation parameters.

End-to-End Enterprise CI/CD Pipeline:

Developer Pushes Code → GitHub Repository → Jenkins Hook → Build Phase → Automated Test → Package Artifacts → Deploy to Target → Environment Verification

Industry-Standard Jenkins Plugins

Plugins are critical components required to shape pipeline behaviors. The following tool sets form the baseline toolkit utilized across production environments:

- **Git Plugin:** Provides foundational connectivity to access, poll, and pull down structural repository branches.
- **Pipeline Plugin:** Enables writing complex custom workflows and multi-step orchestration via standard Domain Specific Language (DSL).
- **Docker Plugin:** Enables dynamically provisioning ephemeral container-based build executors and packaging applications.
- **SSH Plugin:** Facilitates executing remote tasks and configurations securely on decoupled application servers over SSH connections.
- **Email Extension Plugin:** Supplies customizable alert delivery patterns, build failures status notifications, and metric dispatches.
- **Blue Ocean Plugin:** Renders a modern, visual UX layer detailing real-time workflow status mapping and diagnostics.
- **Maven Integration Plugin:** Optimizes native project compilation pipelines for Java-centric applications.

4. Jenkins Installation on AWS EC2

This step-by-step procedural layout governs the structural provisioning and deployment of a production-ready Jenkins server instance onto an AWS EC2 instance running Red Hat Enterprise Linux.

1. **Establish Connection:** Securely connect to your provisioned RHEL 10 EC2 instance using a designated SSH client or terminal handler.

- 2. Install Java Runtime Environment:** Jenkins requires Java to execute. Deploy the open-source Java 21 Development Kit (OpenJDK).
- 3. Append official Jenkins Repository:** Configure tracking targets by safely fetching the stable upstream release repository endpoints.
- 4. Import GPG Validation Keys:** Securely import the authentic community GPG key to verify package sign-offs before extraction.
- 5. Execute Package Installation:** Trigger system dnf utilities to successfully install the official Jenkins binary package.
- 6. Manage Core System Services:** Enable automation daemons and start up the primary background runtime process.
- 7. Configure Security Infrastructures:** Open custom inbound firewall tracking paths (Port 8080) within the cloud Security Group configurations.
- 8. Access Graphical Setup UI:** Navigate a clean web browser to your instance's address to start the initialization phase.
- 9. Unlock Administrator Access:** Access and extract the secure random passphrase generated within the system core logs to unlock admin access.
- 10. Finalize Basic Configurations:** Install the core set of community recommended extensions and declare primary system superuser credentials.

Sequential Deployment Commands

Step 1: Install OpenJDK 21 Core Environment Dependencies

```
sudo dnf install java-21-openjdk java-21-openjdk-devel -y  
java -version
```

Step 2: Add Jenkins Package Repository Reference Path

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

Step 3: Import Official Repository Integrity GPG Keys

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

Step 4: Execute Stable Package Installation via DNF Engine

```
sudo dnf install jenkins -y
```

Step 5: Initialize, Enable, and Inspect Jenkins Daemon Service

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins
```

Step 6: Network Port Verification & Connectivity Inspection

```
ss -tulpn | grep 8080
```

Step 7: Retrieve Secure Root Password to Unlock Initial Access

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

AWS Cloud Network Firewall Rules (Security Group)

To allow incoming external browser connectivity requests to reach the server dashboard, a custom inbound protocol configuration must be defined within your AWS VPC instance rules:

Rule Traffic Type	Custom TCP Rule
Destination Port	8080 (Default Jenkins Port Location)
Allowed IP Range Source	0.0.0.0/0 (Global Access - Recommended only for temporary lab testing)

Initializing Dashboard UI Configurations

- 1. Open an active web browser window and navigate to the deployment endpoint format:

```
http://<YOUR_EC2_PUBLIC_IP>:8080
```

2. Copy the complex alpha-numeric string returned via step 7's file check, paste it inside the 'Administrator Password' form block, and execute validation.
3. Choose the option to 'Install Suggested Plugins' to build your baseline tooling structure.
4. Provision a unique administrative profile username, specify a strong passphrase, and complete the configuration wizard to access your landing hub dashboard.

Output Pictures



